

State Space Models for Spatial Time Series

Anastasia Voznyuk

2025

Task: we have observations in time series $Y_t(u)$ in the form of curves or surfaces over state space $\mathcal{S}$.

Function as time series

$$Y_t(u) = X_t(s) + \epsilon(s) = \sum_{k=1}^K c_{t,k} \phi_k(u)$$

where $\phi_k(u)$ are basis functions (B-splines, Fourier, FPCA eigenfunctions) and $c_{t,k}$ are coefficients describing each function

Usually we want our $Y_t(s)$ to be stationary. and continous on $\mathcal{S}$.

What we want: We want to model latent temporal dynamics, how the pattern of our curves changes over time.

Why State-Space Models?

We use SSM for hidden processes that evolve in time:

Linear-Gaussian State-Space Model

$$\begin{aligned}x_{t+1} &= F_t x_t + w_t, & w_t &\sim \mathcal{N}(0, Q_t) \\ y_t &= H_t x_t + v_t, & v_t &\sim \mathcal{N}(0, R_t)\end{aligned}$$

In FDA setting, y_t becomes a function $Y_t(u) = \sum_{k=1}^K c_{t,k} \phi_k(u)$.

What we can do is model basis coefficients $c_t = (c_{t,1}, \dots, c_{t,K})^\top$ with an SSM:

SSM Model

$$c_{t+1} = F_t c_t + w_t, \quad Y_t(u) = \Phi(u)^\top c_t + v_t(u)$$

Thus we can connect FDA and Time-Series Modeling:

- Kalman filtering is similar functional smoothing in time
- Switching/Nonlinear SSMs is similar to functional regimes and shape changes.

Spatially Coupled Linear-Gaussian SSM

When we add variation across locations:

$$Y_t(s, u) = \Phi(u)^\top c_t(s) + v_t(s, u)$$

we get dynamic model

$$c_{t+1}(s) = F_t c_t(s) + \epsilon_t(s)$$

Then, to obtain spatial coupling we add covariance kernel or Gaussian Markov Random Field (GMRF):

$$\text{Cov}(\epsilon_t(s), \epsilon_t(s')) = \sigma^2 \exp(-\|s - s'\|/\rho)$$

We get Linear dynamics over time, Gaussian noise.

What that gives us:

- Capture smooth temporal evolution of functional shapes.
- Propagate spatial correlations through time.
- We do the inference with Kalman filtering with Kronecker algebra or SPDE approximations.

NB: We can also embed spatial structure by making $Q = \Sigma_t \otimes \Sigma_s$.

Dynamic Linear Model

This is a parameterized and interpretable version of the linear-Gaussian model. We decompose spatial series into level/trend components:

Local Linear Trend Model

$$\mu_{t+1}(s) = \mu_t(s) + \beta_t(s) + w_{t,\mu}(s)$$

$$\beta_{t+1}(s) = \beta_t(s) + w_{t,\beta}(s)$$

$$Y_t(s) = \mu_t(s) + \epsilon_t(s),$$

where $\mu_t(s)$ – level, $\beta_t(s)$ – slope, and usually is a random walk.

- Estimated via Maximum Likelihood (EM algorithm);
- Treated as random and estimated in a Bayesian framework (e.g. MCMC or conjugate updates)
- Implemented in Bayesian hierarchical SSMs.

Nonlinear / Non-Gaussian Spatio-Temporal SSMs

If our spatial dynamic is non-linear, then we express this non-linearity with functions f and g .

$$\begin{aligned}x_{t+1}(s) &= f_t(x_t(\cdot), s) + \eta_t(s), \quad \eta_t(s) \sim \mathcal{N}(0, Q_t(s, s')) \\ y_t(s) &\sim p(y_t(s) | g_t(x_t(s))),\end{aligned}$$

where

- $x_t(s)$: latent state field on spatial domain $\mathcal{S}$
- $y_t(s)$: observed data (possibly discrete or counts)
- $f_t(\cdot)$: state transition / evolution function
- $g_t(\cdot)$: observation (emission) function, **link the latent field to data**, usually Poisson, Bernoulli and etc.
- $Q_t(s, s')$: spatial covariance kernel (smoothness) to define how we **interact** in space
- $p(y_t(s) | \cdot)$: likelihood (may be non-Gaussian)

Usecases for Nonlinear / Non-Gaussian Spatio-Temporal SSMs

We can use this model for various use-cases:

- For physical dynamics (continuous-space models) in geophysical or epidemiological settings;
- For statistical nonlinearities: $f_t(x_t) = Ax_t + Bh(x_t)$
- In Extended / Unscented KF or Particle Filter.

Regime-Switching SSM

We introduce a discrete latent variable $S_t \in \{1, \dots, M\}$, representing the regime or mode active at time t , that allow to switch between distinct dynamic regimes (e.g., “wet” vs. “dry”)

$$\begin{aligned}x_{t+1}(s) \mid S_t = m &= F^{(m)}x_t(s) + \eta_t^{(m)}(s) & \eta_t^{(m)}(s) &\sim \mathcal{N}(0, Q^{(m)}) \\y_t(s) \mid S_t = m &= H^{(m)}x_t(s) + v_t^{(m)}(s) & v_t^{(m)}(s) &\sim p^{(S_t)}(\cdot),\end{aligned}$$

where

- $F^{(m)}, H^{(m)}$ is regime-specific transition and observation functions
- $Q^{(m)}, R^{(m)}$ are regime-specific covariance matrices
- S_t is a hidden Markov Chain, s.t $S_t \sim \text{Categorical}(P_{S_t})$

Properties:

- Mixture of spatial SSMs weighted by regime probabilities.
- Inference: Switching Kalman Filter / EM.
- Used often for nonstationary or clustered spatial behavior.

Latent-Factor / Low-Rank Dynamic SSM

Let us scale up to hundreds or thousands of spatial location. We model the high-dimensional field as driven by a small number $f_t \in \mathbb{R}^r$ latent dynamic factors:

$$f_{t+1} = Af_t + \eta_t, \quad \eta_t \sim \mathcal{N}(0, Q)$$
$$Y_t(s) = \Lambda(s)f_t + v_t(s)$$

- The high-dimensional data is generated by a few latent dynamic factors
- Each factor corresponds to a spatial or functional pattern (a “mode”).
- (s) define how each site/curve participates in those patterns.
- The factors evolve dynamically over time via A .

NB: the observed spatial field at time t is a linear combination of spatial basis functions (s) weighted by evolving coefficients f_t

Code for Nonlinear

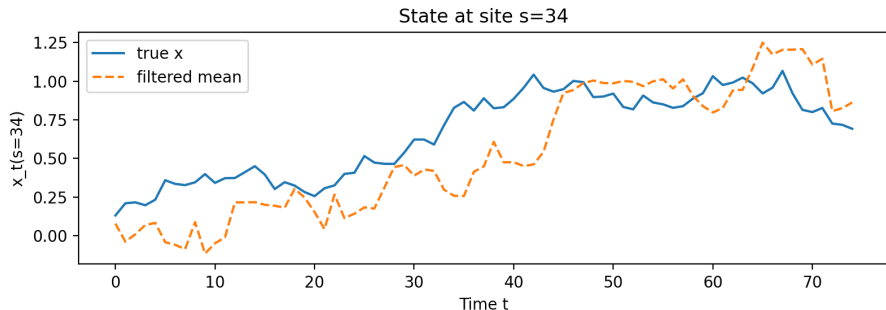
Model:

- 1 Latent field $x_t(s)$ on a 1-D grid evolves with logistic growth + diffusion (nonlinear), plus Gaussian process noise.

$$x_{t+1} = x_t + rx_t\left(1 - \frac{x_t}{K}\right) + \text{noise} \quad (1)$$

where r and K are constants (growth rate and capacity)

- 2 Observations are **Poisson counts** with log-link $\lambda_t(s) = \exp(x_t(s) + b)$
- 3 We also add to Equation 1 smoothing over space.



We use Bootstrap Particle Filter (variant of Non-linear Spatio-Temporal SSM) to infer the latent field $x_t(s)$.

Steps in the algorithm:

- 1 Initialize particles $x_0(i)$;
- 2 Propagate them via the nonlinear logistic-diffusion dynamic;
- 3 Weight them using the Poisson observation likelihood:

$$w_t^{(i)} \propto p(Y_t | x_t^{(i)})$$

- 4 Resample particles with bootstrap;
- 5 Estimate posterior means and log-likelihood;